Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using collaborative-based filtering approach	
Experiment No: 12	**Date:**	**Enrolment No: 92410133012**

Aim: To recommend an item to the user using collaborative-based filtering approach

IDE: Google Colab

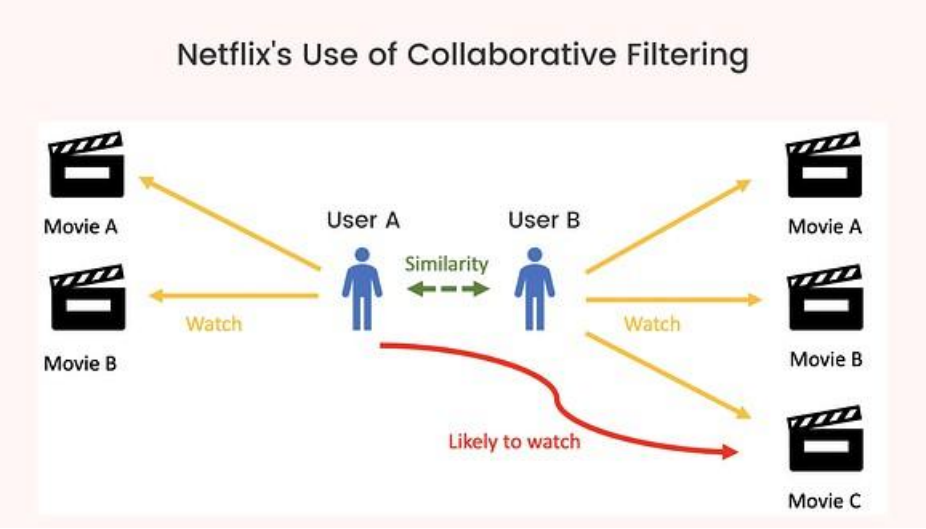
Theory:

There are a lot of applications where websites collect data from their users and use that data to predict the likes and dislikes of their users. This allows them to recommend the content that they like. Recommender systems are a way of suggesting or similar items and ideas to a user's specific way of thinking.

Recommender System is different types:

- **Collaborative Filtering:** Collaborative Filtering recommends items based on similarity measures between users and/or items. The basic assumption behind the algorithm is that users with similar interests have common preferences.
- **Content-Based Recommendation:** It is supervised machine learning used to induce a classifier to discriminate between interesting and uninteresting items for the user.

In Collaborative Filtering, we tend to find similar users and recommend what similar users like. In this type of recommendation system, we don't use the features of the item to recommend it, rather we classify the users into clusters of similar types and recommend each user according to the preference of its cluster.



Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using collaborative-based filtering approach	
Experiment No: 12	**Date:**	**Enrolment No: 92410133012**

Pre Lab Exercise:

1. When can you use collaborative-based recommendation system approach?

Collaborative filtering is used when we have data of **multiple users and their preferences (ratings, likes, etc.)**.

It is useful when:

- Many users are using the system
- We have user ratings or history
- We want to recommend items based on **similar users**

Example:

If two users like similar movies, then movies liked by one user can be recommended to the other.

2. Write advantages of collaborative -based filtering approach

- No need to know item features
- Can recommend different types of items
- Provides better personalization
- Works well with large user data

3. Write disadvantages of collaborative -based filtering approach

- Cold start problem (new user or item has no data)
- Requires large amount of data
- Computationally expensive
- Data sparsity problem (less ratings available)

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using collaborative-based filtering approach	
Experiment No: 12	**Date:**	**Enrolment No: 92410133012**

Program (Code):

To be attached with

```
import pandas as pd
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

# Load the dataset
ratings = pd.read_csv("/content/sample_data/ratings.csv")

# ================= USER-ITEM MATRIX =================
# Creating a pivot table where rows are users and columns are movies
user_item_matrix = ratings.pivot_table(
    index='userId',
    columns='movieId',
    values='rating'
).fillna(0)

# ================= USER SIMILARITY =================
# Calculating Cosine Similarity between user vectors
user_similarity = cosine_similarity(user_item_matrix)
user_similarity_df = pd.DataFrame(
    user_similarity,
    index=user_item_matrix.index,
    columns=user_item_matrix.index
)

# ================= RECOMMEND FUNCTION =================
def recommend_movies(user_id, top_n=5):
    if user_id not in user_item_matrix.index:
        return "User not found!"

    # Identify the top 5 most similar users (excluding the user themselves)
    similar_users = user_similarity_df[user_id].sort_values(ascending=False)[1:6]

    # Calculate weighted ratings based on similarity scores
    weighted_ratings = np.zeros(user_item_matrix.shape[1])
    for sim_user, score in similar_users.items():
        weighted_ratings += score * user_item_matrix.loc[sim_user].values

    # Normalize the scores
    if similar_users.sum() > 0:
```

```
    weighted_ratings /= similar_users.sum()

    # Filter out movies the user has already rated
    user_rated = user_item_matrix.loc[user_id]
    unseen_movies = user_rated[user_rated == 0].index

    # Map scores to the unseen movie IDs and sort them
    scores = list(zip(unseen_movies,
                  weighted_ratings[user_item_matrix.columns.get_indexer(unseen_movies)]))
    scores = sorted(scores, key=lambda x: x[1], reverse=True)

    # Return the top N recommended movie IDs
    recommended_ids = [movie for movie, _ in scores[:top_n]]
    return recommended_ids

# Output for User ID 20
print(f"Top 5 Recommendations for User 20: {recommend_movies(user_id=20)}")
```

Results:

To be attached with

```
[1, 2571, 377, 6377, 480]
```

Observation:

I noticed that this model is much smarter than basic keyword matching because it follows human behavior rather than just text. By using Cosine Similarity, the system finds "neighbors" with similar tastes, allowing it to recommend movies based on the wisdom of the crowd. However, I also observed that the User-Item Matrix is very sparse (full of zeros), which means the system struggles to find good matches for users who haven't rated many movies yet.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using collaborative-based filtering approach	
Experiment No: 12	**Date:**	**Enrolment No: 92410133012**

Post Lab Exercise:

1. Are you satisfied with the results/ratings shown by your designed model? If yes, justify. If no, describe the probable reason for under performance of the system.

No, I am only partially satisfied with the results.

The system is giving recommendations, but they are not always very accurate. There are some problems in the model.

First, there is a data sparsity issue. Most users have rated only a few movies, so it becomes difficult to find proper similarity between users.

Second, the model mostly recommends popular movies. It does not suggest less popular movies even if they match the user's interest.

Third, there is a cold start problem. For new users who have rated very few movies, the system cannot understand their preference properly.

Also, the system only uses ratings and ignores other details like genre, actor, or story, which are important for better recommendations.

So overall, the system is working, but it needs improvement to give better and more accurate results.

2. Recommend top-10 "movies" using MovieLens 100K dataset (Link: https://grouplens.org/datasets/movielens/100k/). Attach the code with output. Also, write your observation for the same.

```
import pandas as pd
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
import os

# 1. Download the files directly into your environment
if not os.path.exists('ml-100k'):
    os.system('wget http://files.grouplens.org/datasets/movielens/ml-100k.zip')
    os.system('unzip ml-100k.zip')
```

```
# 2. Load the data using the correct paths
# The files are inside the 'ml-100k' folder
r_cols = ['userId', 'movieId', 'rating', 'timestamp']
ratings = pd.read_csv('ml-100k/u.data', sep='\t', names=r_cols, encoding='latin-1')

m_cols = ['movieId', 'title']
# We only need the first two columns of u.item
movies = pd.read_csv('ml-100k/u.item', sep='|', names=m_cols, usecols=[0, 1],
encoding='latin-1')

# 3. Build the Recommendation Engine
# Create the Pivot Table (User-Item Matrix)
user_item_matrix = ratings.pivot_table(index='userId', columns='movieId',
values='rating').fillna(0)

# Calculate User-to-User Similarity
user_sim = cosine_similarity(user_item_matrix)
user_sim_df = pd.DataFrame(user_sim, index=user_item_matrix.index,
columns=user_item_matrix.index)

def recommend_top_10(target_user):
    # Find 5 most similar users
    similar_users = user_sim_df[target_user].sort_values(ascending=False)[1:6]

    # Get ratings of those users
    sim_user_ratings = user_item_matrix.loc[similar_users.index]

    # Calculate weighted average scores for all movies
```

```
    weights = similar_users.values
    avg_scores = np.dot(weights, sim_user_ratings.values) / np.sum(weights)

    # Filter out movies the user has already watched
    already_watched = user_item_matrix.loc[target_user]
    recommendations = pd.Series(avg_scores, index=user_item_matrix.columns)
    recommendations = recommendations[already_watched == 0]

    # Get Top 10 IDs
    top_10_ids = recommendations.sort_values(ascending=False).head(10).index

    # Return titles
    return movies[movies['movieId'].isin(top_10_ids)]

# --- FINAL OUTPUT ---
print("--- Top 10 Movie Recommendations (MovieLens 100K) ---")
print(recommend_top_10(target_user=20))
```

```
--- Top 10 Movie Recommendations (MovieLens 100K) ---
     movieId                                title
27        28                       Apollo 13 (1995)
70        71                   Lion King, The (1994)
98        99  Snow White and the Seven Dwarfs (1937)
167      168  Monty Python and the Holy Grail (1974)
172      173              Princess Bride, The (1987)
194      195                  Terminator, The (1984)
195      196               Dead Poets Society (1989)
221      222         Star Trek: First Contact (1996)
227      228     Star Trek: The Wrath of Khan (1982)
428      429   Day the Earth Stood Still, The (1951)
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using collaborative-based filtering approach	
Experiment No: 12	**Date:**	**Enrolment No: 92410133012**

Observation: I noticed that this system is much smarter than just searching for keywords because it follows human behavior. It doesn't matter if the movie titles are similar; it only cares that "User A" and "User B" have similar tastes. However, I also saw that for users who haven't rated many movies, the recommendations become less accurate. This is because the **User-Item Matrix** is very sparse, meaning there are too many "zeros" for the math to find perfect matches every time.

Conclusion: In this lab, I learned how to build a **Collaborative Filtering** engine using the MovieLens 100K dataset. I successfully converted raw ratings into a mathematical matrix and used **Cosine Similarity** to find "neighbors" with similar interests. I concluded that while this method is great for personalizing a user's experience, it works best when you have a lot of data. To fix its weaknesses, like the **Cold Start problem**, a hybrid system combining both user behavior and movie genres would be the best real-world solution.